SAPIENZA
UNIVERSITÀ DI ROMA

Laboratorio di Ingegneria Informatica

Presentazione Esonero

Prof. Roberto **Navigli**
Dott. Domenico **Fedele**

Progetto

Obiettivo complessivo

- Costruire una **pipeline end-to-end** per **acquisire** e **analizzare documenti** da sorgenti **web** eterogenee
- **Applicazione reale**: il vostro lavoro contribuirà direttamente alla versione chatbot dell'LLM nazionale **Minerva** (sviluppato da **Sapienza NLP** e **Babelscape**)
  - Stiamo per lanciare una versione che cerca sul Web e recupera i risultati dal motore di ricerca, li parsa e li utilizza per dare una risposta aggiornata
- E' una bella occasione per lavorare su un progetto concreto nel campo dell'Intelligenza Artificiale, con un impatto reale e immediato

SAPIENZA NLP + Babelscape → minerva

Progetto

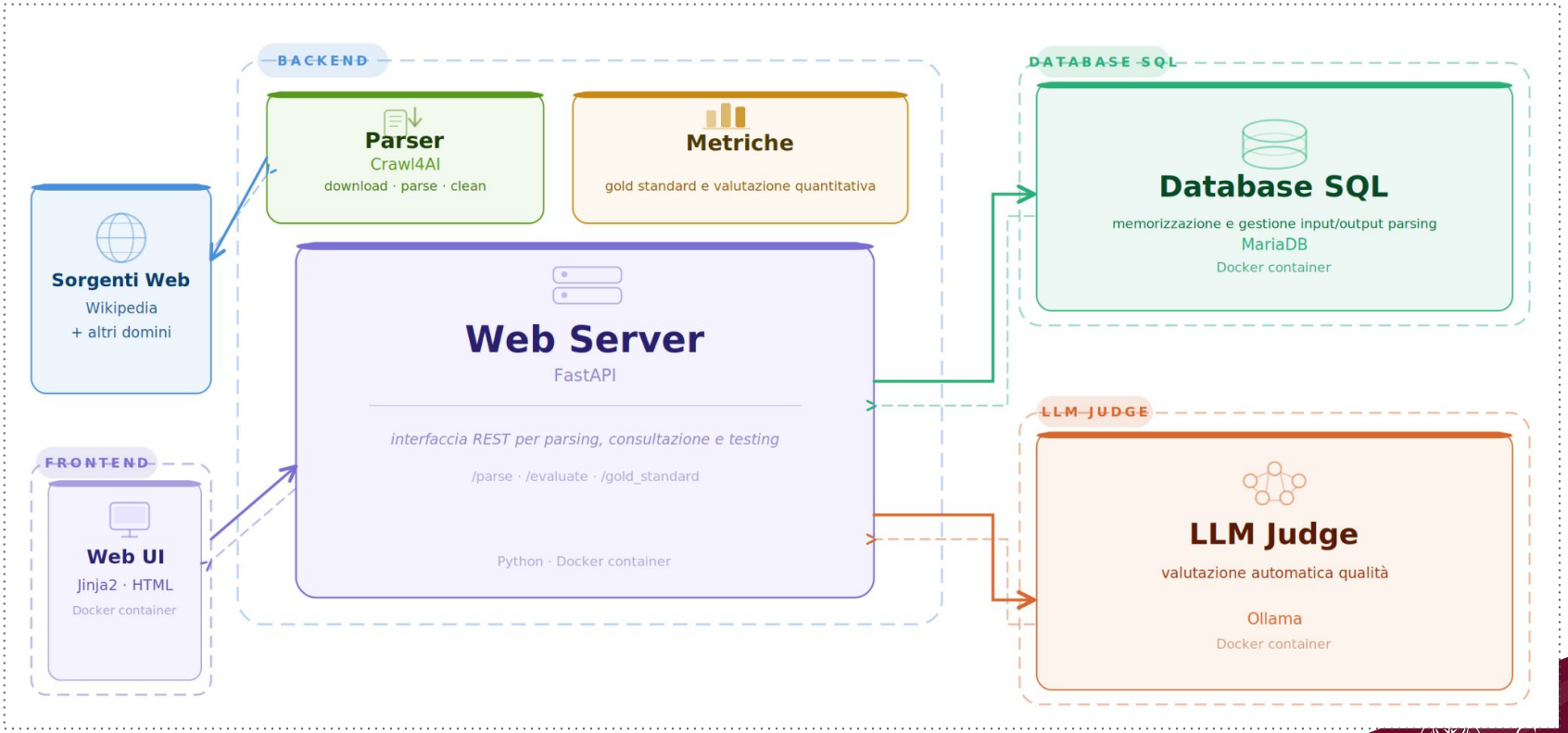
Funzionalità del sistema

- **Acquisizione** di **documenti** da sorgenti **web** eterogenee
- **Estrazione** del **testo pulito** dai documenti acquisiti tramite **parsing automatico**
- **Archiviazione** strutturata dei documenti in un **database relazionale**
- **Valutazione automatica** della **qualità** del testo
- **Giudizio qualitativo** aggiuntivo tramite un **LLM**
- Interfaccia **REST API** e **Web UI** per la consultazione, il testing e la gestione della pipeline
- **Architettura multi-container** orchestrata con Docker Compose (backend, frontend, database, LLM)

SAPIENZA UNIVERSITÀ DI ROMA

Progetto

Architettura del sistema



SAPIENZA UNIVERSITÀ DI ROMA

Progetto

Formazione dei gruppi

- Gli **esoneri** e/o **progetto finale** si svolgono in **gruppi da 3 persone** (devono essere mantenuti gli stessi gruppi)

- Entro il **9 aprile** è necessario compilare il **form** su **Classroom** per **formalizzare** il **gruppo - un gruppo un solo invio!!!**

- Chi ha **difficoltà** a **trovare** un **gruppo, si presenti il 9 aprile in aula per** un matchmaking.

- Eventuali richieste di **gruppi** con numero **ridotto** di studenti**, ben motivate ed eccezionali, saranno valutate** caso per caso (non implicano una semplificazione del progetto)

SAPIENZA
UNIVERSITÀ DI ROMA

Esoneri

Informazioni generali

- **Consegna:** la consegna per l'esonero 1 è prevista **il giorno <u>27 aprile 2026</u>**
- Il primo esonero viene assegnato oggi. Il secondo esonero concluderà il progetto, con consegna **<u>al massimo 8 giorni prima di ciascun appello (ENTRO SETTEMBRE)</u>**
- **Gli esoneri non sono obbligatori**
  - E' possibile **consegnare** il **progetto completo** in un'unica soluzione al massimo **<u>8 giorni prima di ciascun appello</u>**
  - **Esempio:** appello il 25 dicembre → consegna entro il 17 dicembre (mezzanotte circa)
- **Accesso alla tesi**
  - **<u>Tutti gli studenti possono utilizzare il progetto svolto come tesi triennale</u>**, applicando una personalizzazione che sarà illustrata nelle prossime settimane
  - I migliori **~10 studenti** potranno accedere a un **esonero 2 e progetto di tesi personalizzato** in **NLP**
- **Perché fare l'esonero?** Perché copre una parte sostanziale del lavoro progettuale, è seguito con attenzione dai docenti e offre una base già pronta per la prova finale

SAPIENZA UNIVERSITÀ DI ROMA

Esonero 1

Esonero 1

5 obiettivi

1. **Implementazione iniziale dei parser per i domini web assegnati**
   Parser dedicati per estrarre il testo rilevante dalle pagine dei domini assegnati
2. **Costruzione di un gold standard (GS)**
   Dataset di riferimento costruito manualmente, contenente il testo corretto da confrontare con l'output del parser
3. **Valutazione automatica dei parser con metriche specifiche**
   Misurazione della qualità del parsing tramite metriche quantitative
4. **Esposizione dei dati via API REST**
   Endpoint per parsing, accesso ai dati e valutazione automatica
5. **Web UI** e **Containerizzazione** con **Docker**
   Interfaccia di test e ambiente eseguibile in modo semplice e uniforme

NOTA: Database e LLM as a Judge fanno parte dell'architettura complessiva del progetto, non sono richiesti in questo esonero

SAPIENZA UNIVERSITÀ DI ROMA

Obiettivo 1: Parser per Domini Web

Task

- **Parser**: implementare un parser Python specifico per le pagine di ciascun dominio web assegnato (es. it.wikipedia.org)

- Dato un **URL** proveniente da uno dei **domini assegnati** il parser deve **estrarre** le seguenti **informazioni**:
  - "url": "https//...",
  - "domain": "esempio.it",
  - "title": "Titolo della pagina web",
  - "html_text": "Testo estratto della pagina web grezzo (HTML senza filtri), su cui è possibile rieseguire il parsing per ottere di nuovo parsed_text"
  - "parsed_text": "Testo estratto pulito senza tag HTML o di altro genere, in formato **MARKDOWN**"

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 1: Parser per Domini Web

Libreria per il parsing - Crawl4AI

- Per il **download** e il **parsing** delle **pagine web** potete utilizzare la **libreria** Crawl4AI con **configurazione personalizzata** per ogni **dominio**

- **Crawl4AI** è una **libreria** Python **open-source** progettata per il **web scraping** orientato all'AI

- Trasforma qualsiasi **pagina web** in dati **strutturati** (**Markdown**, JSON, screenshot, HTML) **pronti** per essere consumati da **modelli linguistici** (LLM)

- Sfruttate la libreria per estrarre direttamente il **Markdown** ed eventualmente ripulirlo ulteriormente

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 1: Parser per Domini Web

Crawl4AI - Installazione

- Installare la libreria

  pip install -U crawl4ai
- Setup del browser (Chromium scaricato automaticamente)

  crawl4ai-setup
- Verificare l'installazione
  - **crawl4ai-doctor**
  - **python -c "import crawl4ai; print('OK')"**

- Per eventuali problematiche potete consultare la [documentazione]

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 1: Parser per Domini Web

Crawl4AI - Esempio di download e parsing

- ***BrowserConfig***
  Configura il browser
  (headless, proxy, viewport)

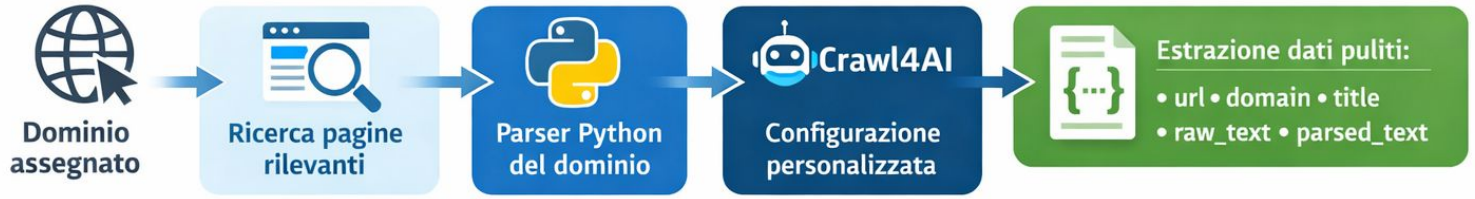
- ***CrawlerRunConfig***
  Configura la singola richiesta. Agire sulla configurazione per migliorare l'output del parsing della libreria

- ***AsyncWebCrawler***
  Il motore del crawler; usato come context manager **apre e chiude il browser in modo sicuro** .arun() visita l'URL e **restituisce il contenuto estratto usando la config scelta**.
  Dovete agire anche sull'output del parsing per migliorare la qualità.

```python
import asyncio
from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig, CacheMode

async def main():

    # configura il browser (headless = senza finestra visibile)
    browser_cfg = BrowserConfig(headless=False)

    # configura la richiesta (BYPASS = scarica sempre dalla rete, ignora cache)
    crawler_cfg = CrawlerRunConfig(cache_mode=CacheMode.BYPASS)

    # apre il browser e lo chiude automaticamente alla fine del blocco
    async with AsyncWebCrawler(config=browser_cfg) as crawler:

        # visita la pagina e aspetta che il crawl sia completato
        result = await crawler.arun(
            url="https://it.wikipedia.org/wiki/Minerva",
            config=crawler_cfg
        )

        # result.success            → True se il crawl è andato a buon fine
        # result.error_message      → messaggio d'errore in caso di fallimento
        #
        # alcuni output disponibili:
        # result.markdown           → testo in Markdown, pronto per LLM e RAG
        # result.cleaned_html       → HTML ripulito da script, stili e rumore
        # result.html               → HTML completo della pagina (non ripulito)
        print(result.markdown)


# avvia il programma asincrono
asyncio.run(main())
```

Obiettivo 1: Parser per Domini Web

Pipeline di Parsing



1. **Selezione** manuale delle **pagine** più **rilevanti** del **dominio** assegnato
2. **Parser Python** dedicato per ciascun dominio
3. **Crawl4AI** con configurazione personalizzata per il dominio specifico (modificare **CrawlerRunConfig** e/o l'output del parsing)
4. Estrazione di **metadati** e **testo pulito** dall'output di Crawl4AI
5. **Output** strutturato: **url, domain, title, html_text, parsed_text**

SAPIENZA
UNIVERSITÀ DI ROMA

Assegnazione dei domini da parsare

- Ogni gruppo dovrà **parsare pagine** provenienti da **domini** web **specifici** (es. it.wikipedia.org , https://www.reddit.com, https://stackoverflow.com e altri)
  - *NOTA: non dovete parsare la home page del sito, ma solo le pagine informative*
- Il **parsing** delle pagine provenienti dal dominio **wikipedia.org** è uguale e **obbligatorio** per tutti (in inglese se, ordinando i cognomi del gruppo in ordine alfabetico, il primo cognome inizia con una lettera "dispari", ovvero A, C, E, G, I, K, M, O, Q, S, U, W, Y; in italiano altrimenti)
- **3 domini aggiuntivi** saranno assegnati **randomicamente** ad ogni gruppo:
  - Almeno **1 su 3** dei domini **aggiuntivi** vanno parsati correttamente per l'esonero
  - **Più domini** riuscirete a parsare correttamente **maggiore** sarà la **valutazione**
  - I **domini aggiuntivi** saranno **assegnati** alla **chiusura** del **form** per la formazione dei **gruppi** (dopo il 9 aprile, potete comunque cominciare con it.wikipedia.org o en.wikipedia.org)
  - Saranno comunicati su **Google Classroom**

SAPIENZA UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard per Domini Assegnati

Cos'è un Gold Standard?

- Un **Gold Standard (GS)** è un insieme di **dati** di riferimento, costruito e verificato **manualmente**, considerato come **corretto**
- Serve a **valutare** la **qualità** dei risultati **prodotti** da un sistema automatico, fornendo un termine di confronto affidabile
- Si **confronta l'output del sistema** con il **GS,** così da misurare in modo oggettivo quanto il risultato prodotto sia corretto

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard per Domini Assegnati

Gold Standard per il parsing di pagine Web

- **Gold Standard (GS)**: per ogni **dominio assegnato**, selezionare almeno **5 URL** rappresentative (pagine di tipo diverso, con contenuti vari, no home page)

- **Per ogni URL:** aprire la pagina nel **browser**, copiare **manualmente** il solo **testo informativo** (no pubblicità, contenuti collaterali o ripetuti in ogni pagina, ecc.)
  - **Includere**: titolo e corpo dell'articolo
  - **NON includere**: menu di navigazione, footer, sidebar, banner pubblicitari

- Utilizzare il **GS** per valutare la **qualità** dei vostri **parser**

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard per Domini Assegnati

Formato del Gold Standard

- Salvare i **GS** in file **JSON, uno per ogni dominio**

- Un **entry** del vostro **GS** contiene le seguenti informazioni:
  - "url": "https//...",
  - "domain": "esempio.it",
  - "title": "Titolo della pagina web",
  - "html_text": "Testo estratto della pagina web grezzo (HTML senza filtri), su cui è possibile rieseguire il parsing per ottere di nuovo *parsed_text* da confrontare con *gold_text*"
  - "gold_text": "Testo estratto pulito senza tag HTML o di altro genere"

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard

Esempio Estrazione Info - Wikipedia

WIKIPEDIA
25 anni di enciclopedia libera

Cerca in Wikipedia | Ricerca

Fai una donazione | Registrati

Indice nascondi

Inizio
Titoli e ruoli
Cultura
Culto
Iconografia
Calculus Minervae
Note
Voci correlate
Altri progetti
Collegamenti esterni

Minerva

79 lingue

Voce | Discussione | Leggi | Modifica | Modifica wikitesto | Cronologia | Strumenti

30 marzo: Wikipedia non può ancora essere venduta

Ci dispiace per i ripetuti tentativi, ma oggi è lunedì, 30 marzo e abbiamo ancora bisogno di aiuto. Se tutti coloro che leggono questo messaggio donassero oggi 2,75 € raggiungeremmo il nostro obiettivo subito. La maggior parte non dona. Wikipedia è gestita da un'organizzazione non-profit, non da un miliardario. Quindi se Wikipedia ti ha dato una conoscenza che vale almeno 2,75 €, per favore dona. Ogni contributo è utile, che siano 2,75 € o 25 €.

Dona 2,75 €

Dona un altro importo

FORSE DOPO | HO GIÀ DONATO | CHIUDI ×

*Disambiguazione – Se stai cercando altri significati, vedi **Minerva (disambigua)**.*

Minerva (in latino *Minerva*) è la divinità romana della lealtà in lotta, delle virtù eroiche, della guerra giusta (guerra per giuste cause o per difesa), della saggezza, delle strategie, ed è riconosciuta anche protettrice degli artigiani.

Le sue origini discendono dagli Etruschi, che molto presto la fusero con Atena, suo corrispettivo nella mitologia greca. Minerva era venerata anche dagli antichi italici, come dimostra il santuario arcaico di Lavinium.

Come per Atena anche per Minerva l'animale sacro è la civetta, talvolta il gufo. Secondo il mito, era figlia di Giove, nata dalla testa di quest'ultimo.

Minerva, dettaglio del *Trionfo della Virtù* di Andrea Mantegna (1499-1502)

Titoli e ruoli [modifica | modifica wikitesto]

Il termine *Minerva* fu probabilmente importato dagli Etruschi che la chiamavano Menrva. I Romani ne confusero il nome straniero con il loro lemma *mens* (mente), dal momento che la dea governava non solo la guerra, ma anche le attività intellettuali

Minerva era figlia di Giove e, dopo l'assimilazione con la greca Atena, il mito della sua nascita la fece frutto dell'unione del re degli dei con la ninfa Meti.

Aspetto nascondi

Modalità compleanno (Baby Globe)
- Disabilitata
- Abilitata

Scopri di più sulla Modalità compleanno

Testo
- Piccolo
- Standard
- Grande

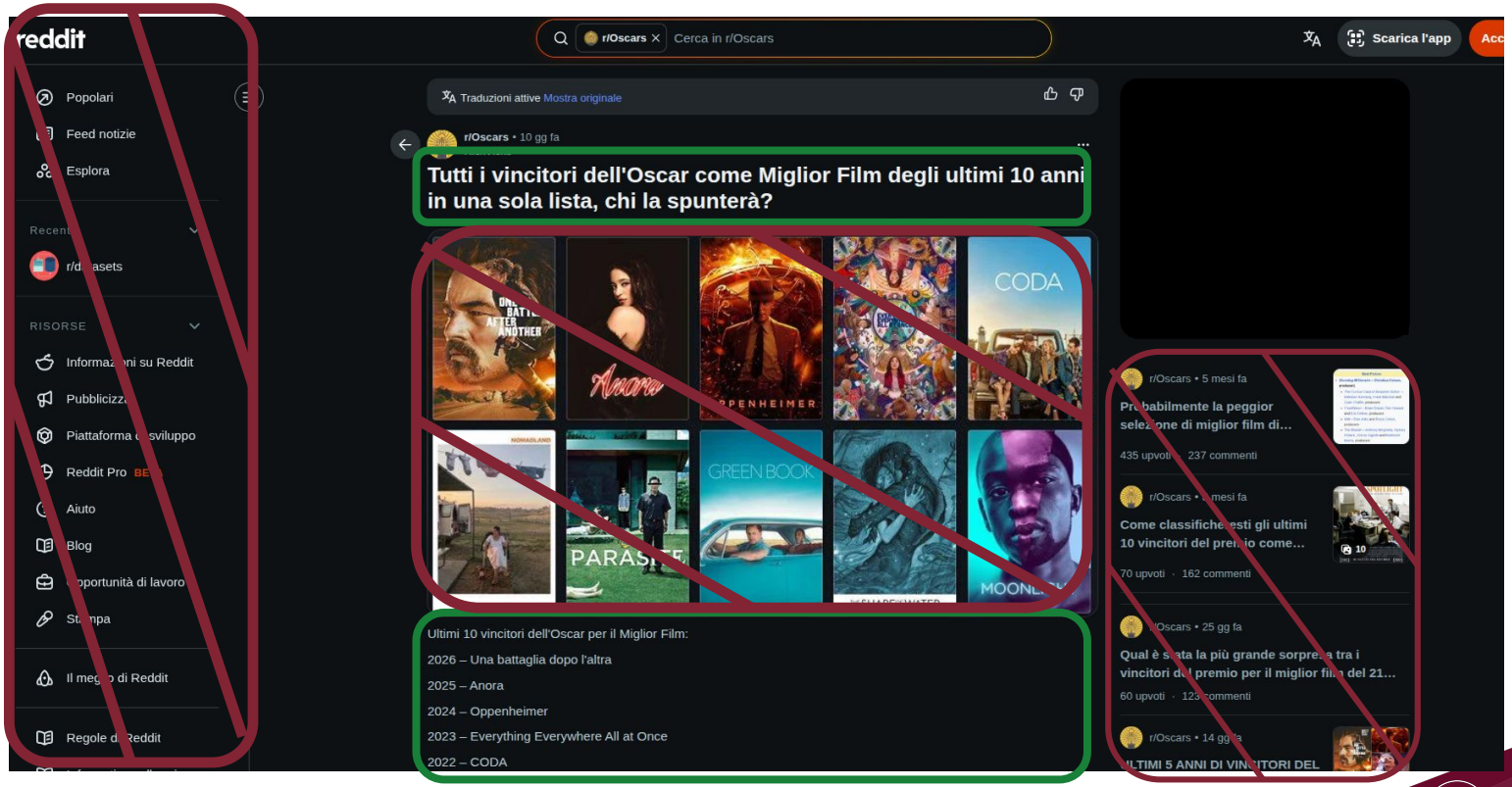
Larghezza
- Standard
- Largo

Colore (beta)
- Automatico
- Chiaro
- Scuro

SAPIENZA UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard

Esempio Estrazione Info – reddit



SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard
Esempi di GS

- Esempi di **elementi** del **GS** per alcune pagine di **Wikipedia**

- Dominio 1: en.wikipedia.org
  - Pagina 1: en.wikipedia.org/wiki/Minerva -> GS
  - Pagina 2: en.wikipedia.org/wiki/BabelNet -> GS
- Dominio 2: it.wikipedia.org
  - Pagina 1: it.wikipedia.org/wiki/Minerva -> GS
  - Pagina 2: it.wikipedia.org/wiki/BabelNet -> GS

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 2: Gold Standard

Trovare pagine interessanti per domini aggiuntivi

- Per trovare pagine interessanti per uno specifico dominio, si possono seguire gli **esempi** presenti nell'**elenco completo dei domini** condiviso su **Classroom**

- Per trovare **altre tipologie** di **pagine** interessanti si può usare **l'operatore** ***"site: <sito web>"*** di Google" per **limitare** la **ricerca** sul **browser** ad un **sito specifico**
- L'idea è cercare contenuti utili e informativi, adatti ad un LLM che cerca sul Web
- **Alcuni Esempi** con Wikipedia in Italiano (*it.wikipedia.org*):
  - "città italiane **site:***it.wikipedia.org*"
  - "energia nucleare **site:***it.wikipedia.org*"
  - "Dante Alighieri **site:***it.wikipedia.org*"

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 3: Valutazione Parser

Valutare Qualità Parsing

- **Valutare** la **qualità** dell'**output** prodotto dai **parser** implementati
- La **valutazione** sarà effettuata tramite **metriche quantitative implementate da voi**, usate come **indicatori** oggettivi della **qualità** del parsing
- I **Gold Standard** (GS) prodotti saranno **utilizzati** come **dataset** di **test** per il confronto con l'output del parser
- Solo **una valutazione** è **obbligatoria** (`token_level_eval`), ma sono **apprezzate** anche eventuali **altre tecniche** di valutazione (sperimentate, cercate online, chiedete agli LLM!)

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 3: Valutazione Parser

Valutare Qualità Parsing

- **Evaluation obbligatoria** da implementare "**token_level_eval**":
  - La valutazione avviene a **livello** di **token**
    - parole separate per spazio in lowercase
    - senza markdown, in questa fase la parte rilevante da valutare è il testo estratto
  - ***tokens_estratti*** = insieme dei token nel testo prodotto dal parser
  - ***tokens_gs*** = insieme dei token nel testo del Gold Standard
  - **Metriche**
    - ***Precision*** = $|tokens_estratti \cap tokens_gs| / |tokens_estratti|$
      Tra i token trovati dal modello, quanti sono davvero corretti?
    - ***Recall*** = $|tokens_estratti \cap tokens_gs| / |tokens_gs|$
      Tra tutti i token corretti possibili, quanti il modello è riuscito a trovare?
    - ***F1*** = $2 \times precision \times recall / (precision + recall)$
      Media armonica tra precision e recall, utile per avere un unico indicatore bilanciato

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 3: Valutazione Parser

Come Interpretare la '*token_level_eval*' F1

Fascia	*Range F1*	*Interpretazione*
Scarso	< 0.6	Il parser perde contenuto rilevante o introduce molto rumore
Medio	0.6 – 0.80	Contenuto principale presente ma con omissioni o boilerplate non rimosso
Buono	> 0.80	Il parser cattura la maggior parte del testo rilevante con poco rumore

NOTA: data la semplicità di questa metrica, "Buono" è condizione necessaria ma non sufficiente per considerare il parser affidabile. Verificate anche manualmente.

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

Overview API REST

- Implementare con **FastAPI** un **WEB Server** per accedere ai seguenti endpoint:
  - **GET /parse**: esegue il parser per un documento di un dominio
  - **POST /parse**: esegue il parser per un documento di un dominio da html diretto
  - **GET /domains**: restituisce la lista dei domini assegnati
  - **GET /gold_standard**: restituisce il gold standard per un documento
  - **GET /full_gold_standard**: restituisce tutto il GS del dominio
  - **POST /evaluate**: dato risultato parsing e gs restituisce le metriche di evaluation
  - **GET /full_gs_eval**: restituisce l'evaluation aggregata su tutto il GS
- Usare **Pydantic** per validazione I/O
- I **formati di input/output** devono **rispettare** esattamente la **specifica**, perché saranno **testati automaticamente**
- **Gestire** esplicitamente **errori** come dominio non supportato e URL irraggiungibile

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

GET /parse

- Seleziona automaticamente il parser corretto in base al dominio dell'URL ed esegue il parsing

- **Input**: **"url": str** (tramite url)
- **Output**: oggetto JSON nel formato indicato

- **Errori**:
  - Se il dominio non è supportato
  - Se l'URL è irraggiungibile

```
// OUTPUT - JSON
{
  "url": "https://...",
  "domain": "esempio.it",
  "title": "Titolo estratto dalla pagina web",
  "html_text": "Html estratto dalla
                pagina web, input del parser",
  "parsed_text": "Testo pulito, output del
                 parser di riferimento, in
                 formato MARKDOWN"
}
```


SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

POST /parse

- Seleziona automaticamente il parser corretto in base al dominio dell'URL ed esegue il **parsing** sull'**html_text** dato in input

- **Input**: **{"url": str, "html_text": str}**
- **Output**: oggetto JSON nel formato indicato

- **Errori**:
  - Se il dominio non è supportato

```
// OUTPUT - JSON
{
  "url": "https://...",
  "domain": "esempio.it",
  "title": "Titolo estratto dalla pagina web",
  "html_text": "Html estratto dalla
                pagina web, input del parser",
  "parsed_text": "Testo pulito, output del
                 parser di riferimento, in
                 formato MARKDOWN"
}
```


SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

Come gestire parsing di HTML diretto

```
SAMPLE_HTML = """<html><body><h1>Minerva</h1><p>Dea romana della sapienza. Corrisponde alla greca
Atena.</p></body></html>"""

async def parse_html():
   browser_config = BrowserConfig(headless=True)
   run_config = CrawlerRunConfig(cache_mode=CacheMode.BYPASS)
   async with AsyncWebCrawler(config=browser_config) as crawler:
       # prefisso "raw:" per indicare che voglio parsare direttamente una stringa HTML
       result = await crawler.arun(
                    url=f"raw:{SAMPLE_HTML}",
                    config=run_config,
       )
   print(result.markdown)
```

- Utilizzando il **prefisso "raw:"** è possibile parsare direttamente una **stringa HTML**, invece di scaricare da un URL.
  Potete comunque usare l'URL per selezionare il parse appropriato.

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

GET /domains

- Restituisce la lista dei domini supportati dal sistema (es. it.wikipedia.org)

- **Input**: nessuno
- **Output**: oggetto JSON nel formato indicato

```
// OUTPUT - JSON
{
  "domains": [
        "esempio_1.it",
        “esempio_2.it",
         ...
 }
```


SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

GET /gold_standard

- Restituisce l'entry del GS per l'URL dato in input.

- **Input**: **"url": str** (tramite url)
- **Output**: oggetto JSON nel formato indicato.

- **Errori**:
  - Se il dominio non è supportato
  - L'URL non è nel GS.

```
// OUTPUT - JSON
{
  "url": "https://...",
  "domain": "esempio.it",
  "title": "Titolo estratto dalla pagina web",
  "html_text": "Html estratto dalla
                pagina web, input del parser",
  "gold_text": "Testo pulito, estratto
                manualmente dalla pagina web,
                no MARKDOWN"
}
```


SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

GET /full_gold_standard

- Restituisce la lista degli elementi di un GS per un dominio specifico

- **Input**: **"domain": str** (tramite url) (lo stesso usato nella lista get domains)
- **Output**: oggetto JSON nel formato indicato.

- **Errori**:
  - Se il dominio non è supportato.

```
// OUTPUT - JSON
 {
“gold_standard”:
  [
        {
        "url": "https://...",
        "domain": "esempio.it",
         "title": "Titolo estratto dalla pagina web",
        "html_text": "Html estratto dalla
                pagina web, input del parser",
        "gold_text": "Testo pulito, estratto
                        manualmente dalla pagina web,
                        no MARKDOWN"
        },
      ...
  ]
}
```


SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

POST /evaluate

- Dato il risultato del parsing di un documento e il corrispettivo testo del GS restituisce le metriche di evaluation (rimuovere il markdown in fase di evaluation, confrontare solo il contenuto)

- **Input**: **{"parsed_text": str, "gold_text": str}**
- **Output**: oggetto JSON nel formato indicato. *Attenzione*: **token_level_eval** con **precision, recall** e **f1** è obbligatorio

```
// OUTPUT - JSON
 {
    “token_level_eval”: {
          "precision": 0.87,
          "recall": 0.91,
           "f1": 0.89,
      },
     “x_eval”:{
      }
}
```


SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

Come gestire confronto tra output in Markdown e Gold Standard

- Pur non coprendo tutti i casi possibili, una possibile soluzione per **rimuovere** il **Markdown** nella valutazione con la ***token_level_eval*** è la seguente:

```
def remove_markdown(md: str) -> str:
    """
    Rimuove il Markdown da una stringa, restituendo solo il testo pulito.
    Usa la libreria mistune per convertire il Markdown in HTML, poi BeautifulSoup per estrarre solo il testo.
    """
    html = mistune.html(md)
    soup = BeautifulSoup(html, "html.parser")
    # rimuove i tag lasciando il testo esattamente in-place (nessun separatore aggiunto, mantiene punteggiatura)
    for tag in soup.find_all(True):
        tag.unwrap()
    text = re.sub(r'[ \t]+', ' ', str(soup))     # collassa spazi orizzontali (non \n)
    text = re.sub(r'\n+', '\n', text)  # collassa nuove linee multiple in una sola
    return text.strip()
```


SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 4: API REST

GET /full_gs_eval

- Restituisce la valutazione complessiva del Gold Standard per il dominio dato in input. Esegue il parsing di tutte le sue entry e aggrega i risultati tramite media delle metriche, calcolate sui singoli elementi del GS (rimuovere il markdown in fase di evaluation, confrontare solo il contenuto)

- **Input**: **"domain": str** (tramite url)
- **Output**: oggetto JSON nel formato indicato.

- **Errori**:
  - Se il dominio non è supportato.

```
// OUTPUT - JSON
 {
    “token_level_eval”: {
          "precision": 0.87,
          "recall": 0.91,
           "f1": 0.89,
      },
     “x_eval”:{
      }
}
```


SAPIENZA UNIVERSITÀ DI ROMA

Obiettivo 5: Web UI e Containerizzazione

Web UI

- **Implementazione** della **Web UI**
  Realizzare **l'interfaccia web** in **HTML**, utilizzando **Jinja2** per il rendering dinamico dei contenuti.
- **Funzionalità principali**
  - Consentire **l'inserimento** di un URL per mostrare un **confronto** tra testo della **pagina web grezzo** (HTML) e **testo pulito** senza tag HTML o di altro genere
  - Mettere a disposizione un **menù a tendina** con tutti gli **URL** presenti nel **Gold Standard** (GS), in modo da eseguire rapidamente la UI
  - Se per l'**URL** selezionato è disponibile il **GS**:
    - **confrontare** il risultato con il testo pulito presente nel GS
    - **mostrare** la **valutazione** tramite le **metriche** già implementate.

SAPIENZA
UNIVERSITÀ DI ROMA

Obiettivo 5: Web UI e Containerizzazione

Containerizzazione con Docker

- **Containerizzazione** del sistema con **Docker**
  Il sistema deve usare **2 Docker** (**Backend & Frontend)**
- **Requisiti**
  - **Backend** containerizzato (FastAPI) sulla porta **8003**
    Un **unico docker** per tutti i parser del server.
  - **Frontend** containerizzato (HTML + Jinja2) sulla porta **8004**
    Un **unico docker** per la UI
  - **docker-compose.yaml** per avviare tutto con un solo comando (come visto a lezione)
  - Usare solo path relativi (nessun path assoluto)
  - Deve funzionare su Ubuntu subito dopo:
    docker compose up --build

SAPIENZA
UNIVERSITÀ DI ROMA

Struttura delle Directory del Progetto

- **docker-compose.yaml**: avvia l'intero sistema con un solo comando
- **backend/**: server FastAPI, parser, modulo di valutazione
- **frontend/**: interfaccia web HTML/Jinja2 con template
- **gs_data/**: Gold Standard JSON per ciascun dominio
- **domains.json**: lista dei domini assegnati
- **report.pdf**: max 2 pagine, redatto in LaTeX

```
progetto/
├── docker-compose.yaml
├── domains.json
├── gs_data/
│   ├── dominio_x_gs.json
│   └── dominio_y_gs.json
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       └── server.py
├── frontend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       └── frontend.py
│   └── templates/
└── report.pdf
```

Test automatico

Test del backend

- Vi forniremo nei prossimi giorni su Google Classroom uno **script** che avvierà un **test automatico** di tutti gli **endpoint** definiti nella specifica del **WEB Server** del progetto

- Dovete **esporre** la **backend** del vostro **progetto** sulla porta **8003**. Lo script di testing, eseguito fuori dai Docker container del vostro progetto, deve poter accedere a quella porta per eseguire il test automatico (non modificare la porta nello script di test)

- Dovete **esporre** la **frontend** del vostro **progetto** sulla porta **8004** (anche se non viene esplicitamente testato)

- Il test deve funzionare subito dopo il comando: **`docker compose up --build`**
  Non ci devono essere passaggi intermedi tra questa inizializzazione automatica e l'esecuzione dello script di test

SAPIENZA UNIVERSITÀ DI ROMA

Test automatico

Test del backend

- Dovete riuscire ad **eseguire** l'intero **script** di **test** senza modifiche arrivando a vedere:
  "ALL TESTS PASS!"
  <u>Indispensabile prima di effettuare la consegna</u>

- **Non** dovete **modificare** lo **script** di **test**, dato che noi lo useremo esattamente così com'è per valutare i progetti

- Se pensate di aver **trovato** un **errore** nello script di test **comunicarlo** tramite **Google Classroom**

SAPIENZA UNIVERSITÀ DI ROMA

Test automatico

Test del frontend

- Ricordate di **testare** voi stessi anche il **frontend** del vostro **progetto**, dopo aver eseguito i test automatici con successo e prima di consegnare
- Il frontend **non viene valutato automaticamente** dallo script di test
- **Non** dovete **consegnare** lo **script di test** insieme al vostro progetto (è uguale per tutti)

SAPIENZA UNIVERSITÀ DI ROMA

Test automatico

Test finali prima della consegna

- Il comando **`docker compose up --build`** deve funzionare su un qualsiasi computer, quindi non si possono usare path assoluti ma solo path relativi.
  Prima di consegnare spostate il vostro progetto in un'altra cartella nel vostro computer e controllare che **`docker compose up --build`** funzioni ancora ***senza cambiare nulla*** per assicurarvi che funzioni anche sul nostro computer che useremo per le correzioni.

- Potete sviluppare su Mac/Windows ma noi eseguiremo il comando **`docker compose up --build`** su Linux (Ubuntu), quindi se state sviluppando su Mac/Windows dovete controllare che questo comando funzioni correttamente anche su **Ubuntu** prima di consegnare (usate un **macchina virutale**, vedere slide successive)

- Ricordate di eseguire anche il test automatico e di testare il frontend dopo ognuno di questi test

SAPIENZA UNIVERSITÀ DI ROMA

Consegna

Consegna

Singolo Zip

- Consegnare un **singolo file zip** che è lo zip della **cartella** del vostro **progetto**, con il nome che include tutte le matricole dei membri del gruppo: **matricola1_matricola2_matricola3_lab_esonero_1.zip**

- La **struttura** delle **directory** all'interno del file zip deve essere in linea con quella **specificata** nelle **slide precedenti**

- Potete usare più file, cartelle e sottocartelle se li reputate utili a organizzare meglio il progetto

SAPIENZA UNIVERSITÀ DI ROMA

Consegna

Docker Compose

- Il vostro **progetto** verrà **eseguito** tramite il comando
  docker compose up --build
  eseguito da terminale nella directory del file
  docker-compose.yaml
- Il file **docker-compose.yaml** contiene le istruzioni necessarie per avviare l'intero progetto con il comando **docker compose up --build**
- I due moduli **frontend** e **backend** sono separati in cartelle diverse, ognuna con il sui file **requirements.txt** e **Dockerfile**, necessari per la costruzione del loro Docker container. Nel modulo **frontend** c'è la cartella **templates** per i file HTML necessari per creare l'interfaccia web.

SAPIENZA UNIVERSITÀ DI ROMA

Consegna

Step da Seguire (1)

- Scaricate l'ISO di **Ubuntu 22.04.5** (desktop) da https://releases.ubuntu.com/jammy/
- Installarlo o come **macchina virtuale** o come **dual boot**. Fate il test finale in un nuovo dual boot o in nuova macchina virtuale, non nel PC in cui avete sviluppato il progetto, questo vi garantisce che funzionerà anche nel mio.
- Tutte le **istruzioni** seguenti vanno **eseguite** dentro la **macchina virtuale** o il dual **boot**.
- Installate **docker** e **docker compose** con le versioni corrette (vedi Google Classroom e form di consegna).
- Navigate da terminale nella **cartella** del vostro **esonero** esattamente come la consegnerete, salvatene anche una copia dato che la dovete consegnare come è prima di **eseguire** i **test** e non dopo

SAPIENZA UNIVERSITÀ DI ROMA

Consegna

Step da Seguire (2)

- Eseguire il comando “**sudo chmod 777 -R .**” per dare i permessi necessari e potenzialmente altre cartelle necessarie.
- Eseguire il comando “**sudo docker compose up --build**” e aspettare che l’inizializzazione termini
- Eseguire il comando lo script di test che deve funzionare dando in output **“ALL TESTS PASS”**
- Consegnare la copia della cartella così come era prima di eseguire il test compressa in un unico file **.zip**

SAPIENZA UNIVERSITÀ DI ROMA

Consegna

Report

- Realizzare un **report** in un file **report.pdf** (anche questo incluso nell'archivio zip) di massimo **2 pagine** (fontsize 11) che descrive il vostro progetto:
  - **Obiettivo del progetto**: Brevissimo riassunto della funzionalità del progetto (1-2 frasi)
  - **Organizzazione del codice**: Come è strutturato il vostro codice e una spiegazione della (singola) responsabilità di ogni modulo
  - **Valutazione parser:** Come sono stati valutati i parser e i risultati ottenuti
  - Siete liberi di aggiungere anche altre sezioni (all'interno del limite totale di due pagine)
  - Il report può essere anche più breve di due pagine (apprezziamo la sintesi)

- Il report deve essere scritto in Latex partendo da questo template online
  - Non includere l'abstract, partite direttamente con la prima **\begin{section}**
  - Decommentare **\aclfinalcopy** eliminando il % davanti a quella riga del file **.tex** per poter mostrare il vostro nome e cognome (altrimenti verrà mostrato "anonymous submission")

- Il progetto va consegnato su **Google Classroom** (tramite l'apposito form)

SAPIENZA UNIVERSITÀ DI ROMA

Valutazione

- L'esonero conterà metà del progetto
- Saranno valutati:
  - L'effettivo funzionamento del progetto (bloccante: non si passa se il progetto non funziona e non passa il test automatico)
  - L'organizzazione del progetto (cartelle, file, ecc.)
  - La qualità e la pulizia del codice (inclusi i commenti e i type hint)
  - La modularità del progetto: la corretta suddivisione in moduli
  - La corretta gestione dei casi limiti o di errore
  - Il report
- Il plagio (sia chi fa copiare che chi copia) porterà all'annullamento del progetto

SAPIENZA
UNIVERSITÀ DI ROMA